marinedustrie.de → Skipily: Shopify-Anbindung ueber Make.com

Kleinschrittige Anleitung. Mit Make-KI: der KI-Szenario-Assistent baut das Geruest, ein KI-Textmodul bereitet jedes Produkt auf (Name kuerzen, HTML entfernen, Kategorie vorschlagen). Stand: Produkt-Sync via x-api-key, lastfest mit Rate-Limit.

---

1. Ueberblick (zwei Richtungen)

- **Produkte:** Shopify (marinedustrie.de) → Make → Skipily-Shop (automatisch bei Anlage/Aenderung).
- **Bestellungen:** Skipily → Make → Shopify (Bestellung/Notiz anlegen). Optional, am Ende beschrieben.

2. Was du brauchst

- Skipily-**Provider-Key** (Provider-Portal, Pro-Tarif) fuer den Header x-api-key.
- Shopify-Admin von marinedustrie.de (fuer Webhooks).
- Ein Make.com-Konto. Fuer das KI-Textmodul ein KI-Zugang in Make (Make AI oder ein verbundenes OpenAI/Claude-Konto).

3. Flow A: Produkte nach Skipily (Schritt fuer Schritt)

Schritt 1: Szenario per Make-KI anlegen

In Make: **Create a new scenario** → oben den KI-Assistenten nutzen („*Create with AI*“ / Prompt-Feld) und folgenden Text einfuegen. Die KI legt Trigger, KI-Modul und HTTP-Modul als Geruest an:

KI (Make): Baue ein Make-Szenario: Trigger = Custom Webhook (empfaengt Shopify products/create und products/update). Danach ein KI-Textmodul, das aus dem Shopify-Produkt den Namen kuerzt, die Beschreibung von HTML befreit und eine Oberkategorie vorschlaegt. Zum Schluss ein HTTP-Modul: POST JSON an eine Skipily-API mit Header x-api-key.

Die KI erstellt einen Entwurf. Die Feinheiten (URL, Header, Mapping) stellst du in den naechsten Schritten ein.

Schritt 2: Trigger = Custom Webhook

- Erstes Modul **Webhooks > Custom webhook** → **Add** → die erzeugte **URL kopieren**.

Schritt 3: Shopify-Webhooks auf die Make-URL

- Shopify-Admin: **Einstellungen** → **Benachrichtigungen** → **Webhooks**.
- Zwei Webhooks (Format **JSON**) auf die Make-URL anlegen: **Produkterstellung** (products/create) und **Produktaktualisierung** (products/update).

Hinweis: Zum Testen in Shopify ein Produkt speichern. In Make unten „Run once“ aktivieren, damit der Webhook die Datenstruktur lernt.

Schritt 4: KI-Modul - Produktdaten aufbereiten

Modul **KI-Text** (Make AI / OpenAI / Claude). Es bekommt das Shopify-Produkt und liefert sauberes JSON (kurzer Name, Beschreibung ohne HTML, passende Kategorie). So wird aus den oft langen marinedustrie-Titeln ein klarer Shop-Eintrag.

KI (Make): Du bekommst ein Shopify-Produkt (Titel, vendor, body_html, product_type, tags). Gib NUR JSON zurueck: {"name": kurzer klarer Produktname ohne Werbefloskeln, "description": Beschreibung als Klartext ohne HTML (max 400 Zeichen), "category": eine der Skipily-Kategorien, die am besten passt}. Waehle die Kategorie aus dieser Liste: .

Hinweis: Die Skipily-Kategorienamen holst du einmalig: GET .../rest/v1/product_categories?select=name_de,slug (Header apikey: anon). Die Liste in den Prompt einsetzen. Ohne Zuordnung bleibt category_id = null (Produkt funktioniert, nur unkategorisiert).

Schritt 5: HTTP-Modul - an Skipily senden

Modul **HTTP > Make a request**. Methode POST. In die Felder die Webhook- und KI-Ausgaben mappen (Make schlaegt Zuordnungen mit *„Map“* teils per KI vor):

```
POST https://vcjwlyqkfkszumdrfvtm.supabase.co/functions/v1/products-api
Header:  Content-Type: application/json
         x-api-key: <DEIN_SKIPILY_PROVIDER_KEY>

{
  "sku":          "{{1.variants[1].sku}}",
  "name":         "{{ bereinigter_name }}",
  "manufacturer": "{{1.vendor}}",
  "description":  "{{ bereinigte_beschreibung }}",
  "price":        {{1.variants[1].price}},
  "currency":     "EUR",
  "ean":          "{{1.variants[1].barcode}}",
  "stock_quantity": {{1.variants[1].inventory_quantity}},
  "in_stock":     {{1.variants[1].inventory_quantity > 0}},
  "weight_kg":    {{1.variants[1].grams / 1000}},
  "images":       ["{{1.image.src}}"],
  "category_id":  {{ skipily_category_id_oder_null }},
  "is_active":    true
}
```

Feld-Zuordnung Shopify → Skipily:

Shopify-Feld	Skipily-Feld (products-api)
title	name (ueber KI gekuerzt)
vendor	manufacturer
body_html	description (ueber KI, HTML entfernt)
variants[].price	price bzw. variants[].price
variants[].sku	sku bzw. variants[].sku
variants[].barcode	ean
variants[].inventory_quantity	stock_quantity / in_stock
variants[].grams	weight_kg (grams/1000)
options + variants	variant_options + variants (1 Zeile je Variante)
image.src / images	images[] (erstes Bild = Titelbild)
product_type / tags	category_id (ueber KI-Vorschlag + Zuordnung)

Schritt 6: Testen

- In Shopify ein Produkt anlegen/aendern → in Make laeuft das Szenario → im Skipily-Shop pruefen.
- Erscheint das Produkt mit sauberem Namen/Preis/Bestand, passt es.

Schritt 7: Scharf schalten

- In Make unten **Scheduling** auf **ON**. Ab jetzt laeuft der Sync automatisch bei jeder Shopify-Aenderung.

4. Varianten (Durchmesser, Laenge, Farbe)

- variant_options = Shopify-Optionen (Name + Werte). variants = eine Zeile je Shopify-Variante mit eigenem Preis/Bestand/SKU.
- variants ersetzt beim POST/PUT ALLE vorhandenen Varianten eines Produkts.

5. Flow B: Bestellungen Skipily -> Shopify (optional)

- Zweites Szenario mit Custom-Webhook. Im Skipily-Provider-Profil die Make-URL als Bestell-Webhook hinterlegen.

- Skipily sendet bei Bestellungen signiert dorthin. In Make daraus eine Shopify-Bestellung (Admin-API) oder Notiz/Mail erzeugen.

6. Gut zu wissen

- **Neu vs. Update:** POST legt an, PUT ?id=<uuid> aktualisiert. Fuer sauberes Update die Skipily-uuid je Shopify-Produkt in einem Make **Data Store** merken.
- **Bestaende:** products/update und die Shopify-Admin-API liefern inventory_quantity; optional zusaetzlich inventory_levels/update.
- **Rate-Limit:** die Skipily-products-api ist lastfest (Schreib-/Leselimit je Provider/IP). Bei Massenimport in Blöcken senden.
- **KI-Kosten im Griff:** das KI-Modul nur im Produkt-Flow nutzen (nicht in Schleifen), kurze Prompts, Ergebnis im Data Store cachen, damit unveraenderte Produkte die KI nicht erneut aufrufen.
- **Uebersetzung:** Skipily uebersetzt Name/Beschreibung on-demand in 6 Sprachen. Die KI in Make muss NICHT uebersetzen, nur bereinigen/kuerzen.